

Common Concurrency Problems

Researchers have spent a great deal of time and effort looking into concurrency bugs over many years. Much of the early work focused on **deadlock**, a topic which we've touched on in the past chapters but will now dive into deeply [C+71]. More recent work focuses on studying other types of common concurrency bugs (i.e., **non-deadlock bugs**). In this chapter, we take a brief look at some example concurrency problems found in real code bases, to better understand what problems to look out for. And thus our central issue for this chapter:

CRUX: HOW TO HANDLE COMMON CONCURRENCY BUGS

Concurrency bugs tend to come in a variety of common patterns. Knowing which ones to look out for is the first step to writing more robust, correct concurrent code.

32.1 What Types Of Bugs Exist?

The first, and most obvious, question is this: what types of concurrency bugs manifest in complex, concurrent programs? This question is difficult to answer in general, but fortunately, some others have done the work for us. Specifically, we rely upon a study by Lu et al. [L+08], which analyzes a number of popular concurrent applications in great detail to understand what types of bugs arise in practice.

The study focuses on four major and important open-source applications: MySQL (a popular database management system), Apache (a well-known web server), Mozilla (the famous web browser), and OpenOffice (a free version of the MS Office suite, which some people actually use). In the study, the authors examine concurrency bugs that have been found and fixed in each of these code bases, turning the developers' work into a quantitative bug analysis; understanding these results can help you understand what types of problems actually occur in mature code bases.

Figure 32.1 shows a summary of the bugs Lu and colleagues studied. From the figure, you can see that there were **105 total bugs**, most of which

Application	What it does	Non-Deadlock	Deadlock
MySQL	Database Server	14	9
Apache	Web Server	13	4
Mozilla	Web Browser	41	16
OpenOffice	Office Suite	6	2
Total		74	31

Figure 32.1: Bugs In Modern Applications

were not deadlock (74); the remaining 31 were deadlock bugs. Further, you can see the number of bugs studied from each application; while OpenOffice only had 8 total concurrency bugs, Mozilla had nearly 60.

We now dive into these different classes of bugs (non-deadlock, deadlock) a bit more deeply. For the first class of non-deadlock bugs, we use examples from the study to drive our discussion. For the second class of deadlock bugs, we discuss the long line of work that has been done in either preventing, avoiding, or handling deadlock.

32.2 Non-Deadlock Bugs

Non-deadlock bugs make up a majority of concurrency bugs, according to Lu’s study. But what types of bugs are these? How do they arise? How can we fix them? We now discuss the two major types of non-deadlock bugs found by Lu et al.: atomicity violation bugs and order violation bugs.

Atomicity-Violation Bugs

The first type of problem encountered is referred to as an atomicity violation. Here is a simple example, found in MySQL. Before reading the explanation, try figuring out what the bug is. Do it!

```
1 Thread 1::
2 if (thd->proc_info) {
3     fputs(thd->proc_info, ...);
4 }
5
6 Thread 2::
7 thd->proc_info = NULL;
```

Figure 32.2: Atomicity Violation (atomicity.c)

In the example, two different threads access the field `proc_info` in the structure `thd`. The first thread checks if the value is non-NULL and then prints its value; the second thread sets it to NULL. Clearly, if the first thread performs the check but then is interrupted before the call to `fputs`, the second thread could run in-between, thus setting the pointer to NULL; when the first thread resumes, it will crash, as a NULL pointer will be dereferenced by `fputs`.

The more formal definition of an atomicity violation, according to Lu et al, is this: “The desired serializability among multiple memory accesses is violated (i.e. a code region is intended to be atomic, but the atomicity is not enforced during execution).” In our example above, the code has an *atomicity assumption* (in Lu’s words) about the check for non-NULL of `proc_info` and the usage of `proc_info` in the `fputs()` call; when the assumption is incorrect, the code will not work as desired.

Finding a fix for this type of problem is often (but not always) straightforward. Can you think of how to fix the code above?

In this solution (Figure 32.3), we simply **add locks around the shared-variable references**, ensuring that when either thread accesses the `proc_info` field, it has a lock held (`proc_info.lock`). Of course, any other code that accesses the structure should also acquire this lock before doing so.

```

1 pthread_mutex_t proc_info_lock = PTHREAD_MUTEX_INITIALIZER;
2
3 Thread 1::
4 pthread_mutex_lock(&proc_info_lock);
5 if (thd->proc_info) {
6     fputs(thd->proc_info, ...);
7 }
8 pthread_mutex_unlock(&proc_info_lock);
9
10 Thread 2::
11 pthread_mutex_lock(&proc_info_lock);
12 thd->proc_info = NULL;
13 pthread_mutex_unlock(&proc_info_lock);

```

Figure 32.3: Atomicity Violation Fixed (**atomicity_fixed.c**)

Order-Violation Bugs

Another common type of non-deadlock bug found by Lu et al. is known as an **order violation**. Here is another simple example; once again, see if you can figure out why the code below has a bug in it.

```

1 Thread 1::
2 void init() {
3     mThread = PR_CreateThread(mMain, ...);
4 }
5
6 Thread 2::
7 void mMain(...) {
8     mState = mThread->State;
9 }

```

Figure 32.4: Ordering Bug (**ordering.c**)

As you probably figured out, the code in Thread 2 seems to assume that the variable `mThread` has already been initialized (and is not NULL);

```

1 pthread_mutex_t mtLock = PTHREAD_MUTEX_INITIALIZER;
2 pthread_cond_t  mtCond = PTHREAD_COND_INITIALIZER;
3 int mtInit
4     = 0;
5
6 Thread 1::
7 void init() {
8     ...
9     mThread = PR_CreateThread(mMain, ...);
10
11     // signal that the thread has been created...
12     pthread_mutex_lock(&mtLock);
13     mtInit = 1;
14     pthread_cond_signal(&mtCond);
15     pthread_mutex_unlock(&mtLock);
16     ...
17 }
18
19 Thread 2::
20 void mMain(...) {
21     ...
22     // wait for the thread to be initialized...
23     pthread_mutex_lock(&mtLock);
24     while (mtInit == 0)
25         pthread_cond_wait(&mtCond, &mtLock);
26     pthread_mutex_unlock(&mtLock);
27
28     mState = mThread->State;
29     ...
30 }

```

Figure 32.5: Fixing The Ordering Violation (**ordering_fixed.c**)

however, if Thread 2 runs immediately once created, the value of `mThread` will not be set when it is accessed within `mMain()` in Thread 2, and will likely **crash with a NULL-pointer dereference**. Note that we assume the value of `mThread` is initially NULL; if not, even stranger things could happen as arbitrary memory locations are accessed through the dereference in Thread 2.

The more formal definition of an order violation is the following: “The desired order between two (groups of) memory accesses is flipped (i.e., *A* should always be executed before *B*, but the order is not enforced during execution)” [L+08].

The fix to this type of bug is generally to **enforce ordering**. As discussed previously, **using condition variables is an easy and robust way to add this style of synchronization** into modern code bases. In the example above, we could thus rewrite the code as seen in Figure 32.5.

In this fixed-up code sequence, we have added a condition variable (`mtCond`) and corresponding lock (`mtLock`), as well as a state variable

(`mtInit`). When the initialization code runs, it sets the state of `mtInit` to 1 and signals that it has done so. If Thread 2 had run before this point, it will be waiting for this signal and corresponding state change; if it runs later, it will check the state and see that the initialization has already occurred (i.e., `mtInit` is set to 1), and thus continue as is proper. Note that we could likely use `mThread` as the state variable itself, but do not do so for the sake of simplicity here. When ordering matters between threads, condition variables (or semaphores) can come to the rescue.

Non-Deadlock Bugs: Summary

A large fraction (97%) of non-deadlock bugs studied by Lu et al. are either atomicity or order violations. Thus, by carefully thinking about these types of bug patterns, programmers can likely do a better job of avoiding them. Moreover, as more automated code-checking tools develop, they should likely focus on these two types of bugs as they constitute such a large fraction of non-deadlock bugs found in deployment.

Unfortunately, not all bugs are as easily fixed as the examples we looked at above. Some require a deeper understanding of what the program is doing, or a larger amount of code or data structure reorganization to fix. Read Lu et al.'s excellent (and readable) paper for more details.

32.3 Deadlock Bugs

Beyond the concurrency bugs mentioned above, a classic problem that arises in many concurrent systems with complex locking protocols is known as **deadlock**. Deadlock occurs, for example, when a thread (say Thread 1) is holding a lock (L1) and waiting for another one (L2); unfortunately, the thread (Thread 2) that holds lock L2 is waiting for L1 to be released. Here is a code snippet that demonstrates such a potential deadlock:

```
Thread 1:                               Thread 2:
pthread_mutex_lock(L1);                  pthread_mutex_lock(L2);
pthread_mutex_lock(L2);                  pthread_mutex_lock(L1);
```

Figure 32.6: Simple Deadlock (`deadlock.c`)

Note that if this code runs, **deadlock does not necessarily occur**; rather, it may occur, if, for example, Thread 1 grabs lock L1 and then a context switch occurs to Thread 2. At that point, Thread 2 grabs L2, and tries to acquire L1. Thus we have a deadlock, as each thread is waiting for the other and neither can run. See Figure 32.7 for a graphical depiction; the presence of a **cycle in the graph** is indicative of the deadlock.

The figure should make the problem clear. How should programmers write code so as to handle deadlock in some way?

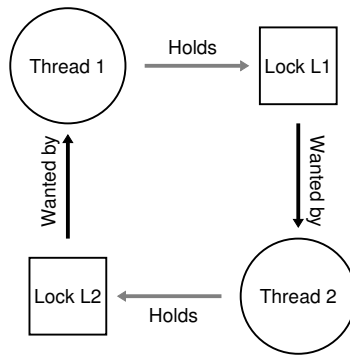


Figure 32.7: The Deadlock Dependency Graph

CRUX: HOW TO DEAL WITH DEADLOCK

How should we build systems to prevent, avoid, or at least detect and recover from deadlock? Is this a real problem in systems today?

Why Do Deadlocks Occur?

As you may be thinking, simple deadlocks such as the one above seem readily avoidable. For example, if Thread 1 and 2 both made sure to grab locks in the same order, the deadlock would never arise. So why do deadlocks happen?

One reason is that **in large code bases, complex dependencies arise between components**. Take the operating system, for example. The virtual memory system might need to access the file system in order to page in a block from disk; the file system might subsequently require a page of memory to read the block into and thus contact the virtual memory system. Thus, **the design of locking strategies in large systems must be carefully done to avoid deadlock in the case of circular dependencies that may occur naturally in the code.**

Another reason is due to the **nature of encapsulation**. As software developers, we are taught to **hide details of implementations** and thus make **software easier to build in a modular way**. Unfortunately, such modularity does not mesh well with locking. As Julia et al. point out [J+08], some seemingly innocuous interfaces almost invite you to deadlock. For example, take the Java Vector class and the method `AddAll()`. This routine would be called as follows:

```
Vector v1, v2;  
v1.AddAll(v2);
```

Internally, because the method needs to be multi-thread safe, locks for both the vector being added to (v1) and the parameter (v2) need to be acquired. The routine acquires said locks in some arbitrary order (say v1 then v2) in order to add the contents of v2 to v1. If some other thread calls `v2.AddAll(v1)` at nearly the same time, we have the potential for deadlock, all in a way that is quite hidden from the calling application.

Conditions for Deadlock

Four conditions need to hold for a deadlock to occur [C+71]:

- **Mutual exclusion:** Threads claim **exclusive control of resources** that they require (e.g., a thread grabs a lock).
- **Hold-and-wait:** Threads **hold resources** allocated to them (e.g., locks that they have already acquired) **while waiting for additional resources** (e.g., locks that they wish to acquire).
- **No preemption:** **Resources** (e.g., locks) **cannot be forcibly removed** from threads that are holding them.
- **Circular wait:** There exists a **circular chain** of threads such that each thread holds one or more resources (e.g., locks) that are being requested by the next thread in the chain.

If any of these four conditions are not met, deadlock cannot occur. Thus, we first explore techniques to *prevent* deadlock; each of these strategies seeks to prevent one of the above conditions from arising and thus is one approach to handling the deadlock problem.

Prevention

Circular Wait

Probably the most practical prevention technique (and certainly one that is frequently employed) is to write your locking code such that you **never induce a circular wait**. The most straightforward way to do that is to provide a **total ordering on lock acquisition**. For example, if there are only two locks in the system (L1 and L2), you can prevent deadlock by **always acquiring L1 before L2**. Such strict ordering ensures that no cyclical wait arises; hence, no deadlock.

Of course, in more complex systems, more than two locks will exist, and thus total lock ordering may be difficult to achieve (and perhaps is unnecessary anyhow). Thus, a **partial ordering** can be a useful way to **structure lock acquisition so as to avoid deadlock**. An excellent real example of partial lock ordering can be seen in the memory mapping code in Linux [T+94] (v5.2); the comment at the top of the source code reveals ten different groups of lock acquisition orders, including simple

TIP: ENFORCE LOCK ORDERING BY LOCK ADDRESS

In some cases, a function must grab two (or more) locks; thus, we know we must be careful or deadlock could arise. Imagine a function that is called as follows: `do_something(mutex_t *m1, mutex_t *m2)`. If the code always grabs `m1` before `m2` (or always `m2` before `m1`), it could deadlock, because one thread could call `do_something(L1, L2)` while another thread could call `do_something(L2, L1)`.

To avoid this particular issue, the clever programmer can use the address of each lock as a way of ordering lock acquisition. By acquiring locks in either high-to-low or low-to-high address order, `do_something()` can guarantee that it always acquires locks in the same order, regardless of which order they are passed in. The code would look something like this:

```
if (m1 > m2) { // grab in high-to-low address order
    pthread_mutex_lock(m1);
    pthread_mutex_lock(m2);
} else {
    pthread_mutex_lock(m2);
    pthread_mutex_lock(m1);
}
// Code assumes that m1 != m2 (not the same lock)
```

By using this simple technique, a programmer can ensure a simple and efficient deadlock-free implementation of multi-lock acquisition.

ones such as “`i_mutex` before `immap_rwlock`” and more complex orders such as “`immap_rwlock` before `private_lock` before `swap_lock` before `i_pages lock`”.

As you can imagine, both total and partial ordering require careful design of locking strategies and must be constructed with great care. Further, ordering is just a convention, and a sloppy programmer can easily ignore the locking protocol and potentially cause deadlock. Finally, lock ordering requires a deep understanding of the code base, and how various routines are called; just one mistake could result in the “D” word¹.

Hold-and-wait

The hold-and-wait requirement for deadlock can be avoided by acquiring all locks at once, atomically. In practice, this could be achieved as follows:

```
1  pthread_mutex_lock(prevention);    // begin acquisition
2  pthread_mutex_lock(L1);
3  pthread_mutex_lock(L2);
4  ...
5  pthread_mutex_unlock(prevention); // end
```

¹Hint: “D” stands for “Deadlock”.

By first grabbing the lock `prevention`, this code guarantees that no untimely thread switch can occur in the midst of lock acquisition and thus deadlock can once again be avoided. Of course, `it requires that any time any thread grabs a lock, it first acquires the global prevention lock`. For example, if another thread was trying to grab locks `L1` and `L2` in a different order, it would be OK, because it would be holding the prevention lock while doing so.

Note that the solution is `problematic` for a number of reasons. As before, encapsulation works against us: when calling a routine, this approach requires us `to know exactly which locks must be held` and to acquire them ahead of time. This technique also is likely to `decrease concurrency as all locks must be acquired early on (at once)` instead of when they are truly needed.

No Preemption

Because we generally view locks as held until `unlock` is called, multiple lock acquisition often gets us into trouble because when waiting for one lock we are holding another. Many thread libraries provide a more flexible set of interfaces to help avoid this situation. Specifically, the routine `pthread_mutex_trylock()` `either grabs the lock (if it is available) and returns success or returns an error code` indicating the lock is held; in the latter case, you can `try again later` if you want to grab that lock.

Such an interface could be used as follows to build a deadlock-free, ordering-robust lock acquisition protocol:

```

1 top:
2   pthread_mutex_lock(L1);
3   if (pthread_mutex_trylock(L2) != 0) {
4     pthread_mutex_unlock(L1);
5     goto top;
6   }
```

Note that another thread could follow the same protocol but grab the locks in the other order (`L2` then `L1`) and the program would still be deadlock free. One new problem does arise, however: `livelock`. It is possible (though perhaps unlikely) that `two threads could both be repeatedly attempting this sequence and repeatedly failing to acquire both locks`. In this case, both systems are running through this code sequence over and over again (and thus `it is not a deadlock`), but `progress is not being made`, hence the name livelock. There are solutions to the livelock problem, too: for example, one could `add a random delay` before `looping back and trying the entire thing over again`, thus decreasing the odds of repeated interference among competing threads.

One point about this solution: it skirts around the hard parts of using a trylock approach. The first `problem` that would likely exist again arises due to encapsulation: if one of these locks is buried in some routine that is getting called, the `jump back to the beginning becomes more complex to implement`. `If the code had acquired some resources (other than L1)`

along the way, it must make sure to carefully release them as well; for example, if after acquiring L1, the code had allocated some memory, it would have to release that memory upon failure to acquire L2, before jumping back to the top to try the entire sequence again. However, in limited circumstances (e.g., the Java vector method mentioned earlier), this type of approach could work well.

You might also notice that this approach **doesn't really add preemption** (the forcible action of taking a lock away from a thread that owns it), but rather uses the trylock approach to allow a developer to back out of lock ownership (i.e., **preempt their own ownership**) in a graceful way. However, it is a practical approach, and thus we include it here, despite its imperfection in this regard.

Mutual Exclusion

The final prevention technique would be to avoid the need for mutual exclusion at all. In general, we know this is difficult, because the code we wish to run does indeed have critical sections. So what can we do?

Herlihy had the idea that one could design various **data structures without locks** at all [H91, H93]. The idea behind these **lock-free (and related wait-free) approaches** here is simple: using powerful hardware instructions, you can build data structures in a manner that does not require explicit locking.

As a simple example, let us assume we have a **compare-and-swap** instruction, which as you may recall is an **atomic instruction** provided by the hardware that does the following:

```
1 int CompareAndSwap(int *address, int expected, int new) {
2     if (*address == expected) {
3         *address = new;
4         return 1; // success
5     }
6     return 0; // failure
7 }
```

Imagine we now wanted to **atomically increment a value by a certain amount**, using compare-and-swap. We could do so with the following simple function:

```
1 void AtomicIncrement(int *value, int amount) {
2     do {
3         int old = *value;
4     } while (CompareAndSwap(value, old, old + amount) == 0);
5 }
```

Instead of acquiring a lock, doing the update, and then releasing it, we have instead built an approach that repeatedly tries to update the value to the new amount and uses the compare-and-swap to do so. In this manner,

no lock is acquired, and **no deadlock** can arise (though **livelock is still a possibility**, and thus a robust solution will be more complex than the simple code snippet above).

Let us consider a slightly more complex example: **list insertion**. Here is code that inserts at the head of a list:

```
1 void insert(int value) {
2     node_t *n = malloc(sizeof(node_t));
3     assert(n != NULL);
4     n->value = value;
5     n->next = head;
6     head = n;
7 }
```

This code performs a simple insertion, but **if called by multiple threads at the “same time”, has a race condition**. Can you figure out why? (draw a picture of what could happen to a list if two concurrent insertions take place, assuming, as always, a malicious scheduling interleaving). Of course, we could solve this by surrounding this code with a lock acquire and release:

```
1 void insert(int value) {
2     node_t *n = malloc(sizeof(node_t));
3     assert(n != NULL);
4     n->value = value;
5     pthread_mutex_lock(listlock); // begin critical section
6     n->next = head;
7     head = n;
8     pthread_mutex_unlock(listlock); // end critical section
9 }
```

In this solution, we are using locks in the traditional manner². Instead, let us try to perform this insertion in a **lock-free manner** simply using the compare-and-swap instruction. Here is one possible approach:

```
1 void insert(int value) {
2     node_t *n = malloc(sizeof(node_t));
3     assert(n != NULL);
4     n->value = value;
5     do {
6         n->next = head;
7     } while (!CompareAndSwap(&head, n->next, n) == 0);
8 }
```

²The astute reader might be asking why we grabbed the lock so late, instead of right when entering `insert()`; can you, astute reader, figure out why that is likely correct? What assumptions does the code make, for example, about the call to `malloc()`?

The code here updates the next pointer to point to the current head, and then tries to swap the newly-created node into position as the new head of the list. However, this will fail if some other thread successfully swapped in a new head in the meanwhile, causing this thread to retry again with the new head.

Of course, building a useful list requires more than just a list insert, and not surprisingly building a list that you can insert into, delete from, and perform lookups on in a lock-free manner is non-trivial. Read the rich literature on lock-free and wait-free synchronization to learn more [H01, H91, H93].

Deadlock Avoidance via Scheduling

Instead of deadlock prevention, in some scenarios deadlock avoidance is preferable. Avoidance requires some global knowledge of which locks various threads might grab during their execution, and subsequently schedules said threads in a way as to guarantee no deadlock can occur.

For example, assume we have two processors and four threads which must be scheduled upon them. Assume further we know that Thread 1 (T1) grabs locks L1 and L2 (in some order, at some point during its execution), T2 grabs L1 and L2 as well, T3 grabs just L2, and T4 grabs no locks at all. We can show these lock acquisition demands of the threads in tabular form:

	T1	T2	T3	T4
L1	yes	yes	no	no
L2	yes	yes	yes	no

A smart scheduler could thus compute that as long as T1 and T2 are not run at the same time, no deadlock could ever arise. Here is one such schedule:

CPU 1	T3	T4
CPU 2	T1	T2

Note that it is OK for (T3 and T1) or (T3 and T2) to overlap. Even though T3 grabs lock L2, it can never cause a deadlock by running concurrently with other threads because it only grabs one lock.

Let's look at one more example. In this one, there is more contention for the same resources (again, locks L1 and L2), as indicated by the following contention table:

	T1	T2	T3	T4
L1	yes	yes	yes	no
L2	yes	yes	yes	no

TIP: DON'T ALWAYS DO IT PERFECTLY (TOM WEST'S LAW)

Tom West, famous as the subject of the classic computer-industry book *Soul of a New Machine* [K81], says famously: "Not everything worth doing is worth doing well", which is a terrific engineering maxim. If a bad thing happens rarely, certainly one should not spend a great deal of effort to prevent it, particularly if the cost of the bad thing occurring is small. If, on the other hand, you are building a space shuttle, and the cost of something going wrong is the space shuttle blowing up, well, perhaps you should ignore this piece of advice.

Some readers object: "This sounds like you are suggesting mediocrity as a solution!" Perhaps they are right, that we should be careful with advice such as this. However, our experience tells us that in the world of engineering, with pressing deadlines and other real-world concerns, one will always have to decide which aspects of a system to build well and which to put aside for another day. The hard part is knowing which to do when, a bit of insight only gained through experience and dedication to the task at hand.

In particular, threads **T1, T2, and T3** all need to grab both locks **L1** and **L2** at some point during their execution. Here is a possible schedule that guarantees that no deadlock could ever occur:



As you can see, **static scheduling** leads to a conservative approach where **T1, T2, and T3** are all run on the same processor, and thus the **total time to complete the jobs is lengthened** considerably. Though it may have been possible to run these tasks concurrently, the fear of deadlock prevents us from doing so, and the cost is performance.

One famous example of an approach like this is **Dijkstra's Banker's Algorithm** [D64], and many similar approaches have been described in the literature. Unfortunately, they are only **useful in very limited environments**, for example, in an **embedded system where one has full knowledge of the entire set of tasks** that must be run and the locks that they need. Further, such approaches can limit concurrency, as we saw in the second example above. Thus, avoidance of deadlock via scheduling is not a widely-used general-purpose solution.

Detect and Recover

One final general strategy is to **allow deadlocks to occasionally occur, and then take some action once such a deadlock has been detected**. For exam-

ple, if an OS froze once a year, you would just reboot it and get happily (or grumpily) on with your work. If deadlocks are rare, such a non-solution is indeed quite pragmatic.

Many database systems employ deadlock detection and recovery techniques. A deadlock detector runs periodically, building a resource graph and checking it for cycles. In the event of a cycle (deadlock), the system needs to be restarted. If more intricate repair of data structures is first required, a human being may be involved to ease the process.

More detail on database concurrency, deadlock, and related issues can be found elsewhere [B+87, K87]. Read these works, or better yet, take a course on databases to learn more about this rich and interesting topic.

32.4 Summary

In this chapter, we have studied the types of bugs that occur in concurrent programs. The first type, non-deadlock bugs, are surprisingly common, but often are easier to fix. They include atomicity violations, in which a sequence of instructions that should have been executed together was not, and order violations, in which the needed order between two threads was not enforced.

We have also briefly discussed deadlock: why it occurs, and what can be done about it. The problem is as old as concurrency itself, and many hundreds of papers have been written about the topic. The best solution in practice is to be careful, develop a lock acquisition order, and thus prevent deadlock from occurring in the first place. Wait-free approaches also have promise, as some wait-free data structures are now finding their way into commonly-used libraries and critical systems, including Linux. However, their lack of generality and the complexity to develop a new wait-free data structure will likely limit the overall utility of this approach. Perhaps the best solution is to develop new concurrent programming models: in systems such as MapReduce (from Google) [GD02], programmers can describe certain types of parallel computations without any locks whatsoever. Locks are problematic by their very nature; perhaps we should seek to avoid using them unless we truly must.

References

- [B+87] “Concurrency Control and Recovery in Database Systems” by Philip A. Bernstein, Vassos Hadzilacos, Nathan Goodman. Addison-Wesley, 1987. *The classic text on concurrency in database management systems. As you can tell, understanding concurrency, deadlock, and other topics in the world of databases is a world unto itself. Study it and find out for yourself.*
- [C+71] “System Deadlocks” by E.G. Coffman, M.J. Elphick, A. Shoshani. ACM Computing Surveys, 3:2, June 1971. *The classic paper outlining the conditions for deadlock and how you might go about dealing with it. There are certainly some earlier papers on this topic; see the references within this paper for details.*
- [D64] “Een algoritme ter voorkoming van de dodelijke omarming” by Edsger Dijkstra. 1964. Available: <http://www.cs.utexas.edu/users/EWD/ewd01xx/EWD108.PDF>. *Indeed, not only did Dijkstra come up with a number of solutions to the deadlock problem, he was the first to note its existence, at least in written form. However, he called it the “deadly embrace”, which (thankfully) did not catch on.*
- [GD02] “MapReduce: Simplified Data Processing on Large Clusters” by Sanjay Ghemawat, Jeff Dean. OSDI ’04, San Francisco, CA, October 2004. *The MapReduce paper ushered in the era of large-scale data processing, and proposes a framework for performing such computations on clusters of generally unreliable machines.*
- [H01] “A Pragmatic Implementation of Non-blocking Linked-lists” by Tim Harris. International Conference on Distributed Computing (DISC), 2001. *A relatively modern example of the difficulties of building something as simple as a concurrent linked list without locks.*
- [H91] “Wait-free Synchronization” by Maurice Herlihy. ACM TOPLAS, 13:1, January 1991. *Herlihy’s work pioneers the ideas behind wait-free approaches to writing concurrent programs. These approaches tend to be complex and hard, often more difficult than using locks correctly, probably limiting their success in the real world.*
- [H93] “A Methodology for Implementing Highly Concurrent Data Objects” by Maurice Herlihy. ACM TOPLAS, 15:5, November 1993. *A nice overview of lock-free and wait-free structures. Both approaches eschew locks, but wait-free approaches are harder to realize, as they try to ensure that any operation on a concurrent structure will terminate in a finite number of steps (e.g., no unbounded looping).*
- [J+08] “Deadlock Immunity: Enabling Systems To Defend Against Deadlocks” by Horatiu Julia, Daniel Tralamazza, Cristian Zamfir, George Candea. OSDI ’08, San Diego, CA, December 2008. *An excellent recent paper on deadlocks and how to avoid getting caught in the same ones over and over again in a particular system.*
- [K81] “Soul of a New Machine” by Tracy Kidder. Backbay Books, 2000 (reprint of 1980 version). *A must-read for any systems builder or engineer, detailing the early days of how a team inside Data General (DG), led by Tom West, worked to produce a “new machine.” Kidder’s other books are also excellent, including “Mountains beyond Mountains.” Or maybe you don’t agree with us, comma?*
- [K87] “Deadlock Detection in Distributed Databases” by Edgar Knapp. ACM Computing Surveys, 19:4, December 1987. *An excellent overview of deadlock detection in distributed database systems. Also points to a number of other related works, and thus is a good place to start your reading.*
- [L+08] “Learning from Mistakes — A Comprehensive Study on Real World Concurrency Bug Characteristics” by Shan Lu, Soyeon Park, Eunsoo Seo, Yuanyuan Zhou. ASPLOS ’08, March 2008, Seattle, Washington. *The first in-depth study of concurrency bugs in real software, and the basis for this chapter. Look at Y.Y. Zhou’s or Shan Lu’s web pages for many more interesting papers on bugs.*
- [T+94] “Linux File Memory Map Code” by Linus Torvalds and many others. Available online at: <http://lxr.free-electrons.com/source/mm/filemap.c>. *Thanks to Michael Wal-fish (NYU) for pointing out this precious example. The real world, as you can see in this file, can be a bit more complex than the simple clarity found in textbooks...*

Homework (Code)

This homework lets you explore some real code that deadlocks (or avoids deadlock). The different versions of code correspond to different approaches to avoiding deadlock in a simplified `vector_add()` routine. See the README for details on these programs and their common substrate.

Questions

1. First let's make sure you understand how the programs generally work, and some of the key options. Study the code in `vector-deadlock.c`, as well as in `main-common.c` and related files.
Now, run `./vector-deadlock -n 2 -l 1 -v`, which instantiates two threads (`-n 2`), each of which does one vector add (`-l 1`), and does so in verbose mode (`-v`). Make sure you understand the output. How does the output change from run to run?
2. Now add the `-d` flag, and change the number of loops (`-l`) from 1 to higher numbers. What happens? Does the code (always) deadlock?
3. How does changing the number of threads (`-n`) change the outcome of the program? Are there any values of `-n` that ensure no deadlock occurs?
4. Now examine the code in `vector-global-order.c`. First, make sure you understand what the code is trying to do; do you understand why the code avoids deadlock? Also, why is there a special case in this `vector_add()` routine when the source and destination vectors are the same?
5. Now run the code with the following flags: `-t -n 2 -l 100000 -d`. How long does the code take to complete? How does the total time change when you increase the number of loops, or the number of threads?
6. What happens if you turn on the parallelism flag (`-p`)? How much would you expect performance to change when each thread is working on adding different vectors (which is what `-p` enables) versus working on the same ones?
7. Now let's study `vector-try-wait.c`. First make sure you understand the code. Is the first call to `pthread_mutex_trylock()` really needed?
Now run the code. How fast does it run compared to the global order approach? How does the number of retries, as counted by the code, change as the number of threads increases?
8. Now let's look at `vector-avoid-hold-and-wait.c`. What is the main problem with this approach? How does its performance compare to the other versions, when running both with `-p` and without it?
9. Finally, let's look at `vector-nolock.c`. This version doesn't use locks at all; does it provide the exact same semantics as the other versions? Why or why not?
10. Now compare its performance to the other versions, both when threads are working on the same two vectors (no `-p`) and when each thread is working on separate vectors (`-p`). How does this no-lock version perform?